

onto a single chip. This effective embedding of device controllers primarily reduces cost and space, making it suitable for embedded systems. In such cases, the buses are integrated into the chip itself. Does this change anything for the drivers, or more generically, on the software front? The answer is, not much—except that the bus drivers corresponding to the embedded device controllers are now developed under the architecture-specific umbrella.

Drivers have two parts

Bus drivers provide hardware-specific interfaces for the corresponding hardware protocols, and are the bottom-most horizontal software layers of an operating system (OS). Over these sit the actual device drivers. These operate on the underlying devices using the horizontal layer interfaces, and hence are device-specific. However, the whole idea of writing these drivers is to provide an abstraction to the user, and so, at the other 'end', these do provide an interface (which varies from OS to OS). In short, a device driver has two parts, which are: a) device-specific, and b) OS-specific. Refer to Figure 2.

The device-specific portion of a device driver remains the same across all operating systems, and is more about understanding and decoding the device data sheets than software programming. A data sheet for a device is a document with technical details of the device, including its operation, performance, programming, etc.—in short a device user manual. Later, I shall show some examples of decoding data sheets as well. However, the OS-specific portion is the one that is tightly coupled with the OS mechanisms of user interfaces, and thus differentiates a Linux device driver from a Windows device driver and from a MacOS device driver.

Verticals

In Linux, a device driver provides a 'system call' interface to the user; this is the boundary line between the so-called kernel space and user-space of Linux, as shown in Figure 2. Figure 3 provides further classification. Based on the OS-specific interface of a driver, in Linux, a driver is broadly classified into three verticals:

- Packet-oriented or the network vertical
- Block-oriented or the storage vertical
- Byte-oriented or the character vertical

The *CPU vertical* and *memory vertical*, taken together with the other three verticals, give the complete overview of the Linux kernel, like any textbook definition of an OS: "An OS performs 5 management functions: CPU/process, memory, network, storage, device I/O." Though these two verticals could be classified as device drivers, where CPU and memory are the respective devices, they are treated differently, for many reasons. These are the core functionalities of any OS, be it a micro-kernel or a monolithic kernel. More often than not, adding code in these

ELECTROMECH CORPORATION

RHCE / RHCSS Training Training by experts



RHCE Exam 26th Nov. 2010

RHCSS RH423 26th Nov. 2010

RHCSS RH333 26th Nov. 2010

Ahmedabad, Baroda, Rajkot—Gujarat

"Come home of Open Source"



Redhat Channel Partner and Training Partner

Ahmedabad: 404, Maulik Arcad, Nr. Mansi
cross Road, Satellite Rd

Baroda: 25, The Emperor, Fatehgunj

Ph.: 079 40027898/9327218003/ 9327247499

Rajkot: 09427576240

[Http://www.electromech.info](http://www.electromech.info)

E-mail: electromech@electromech.info